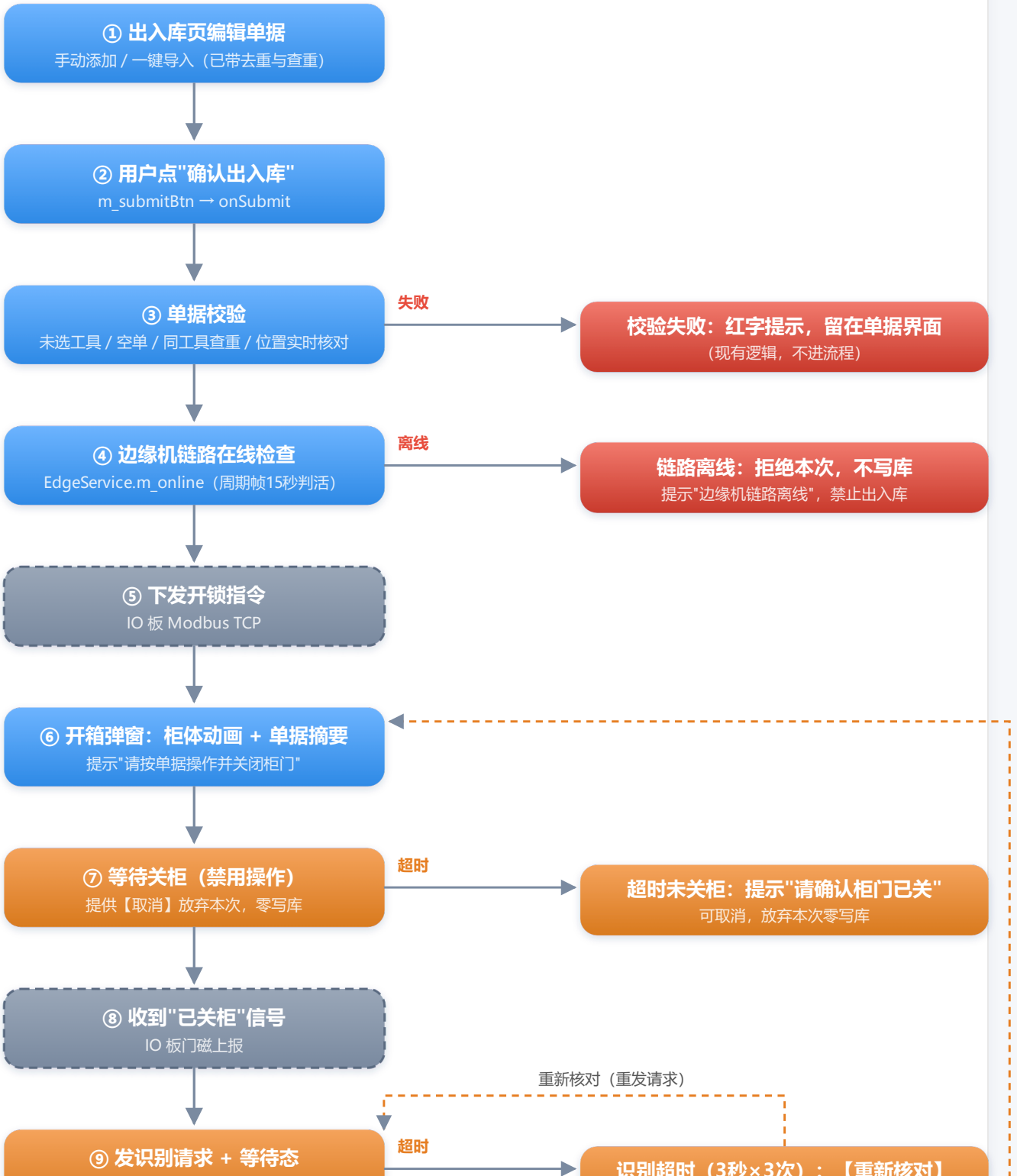
工具出入库：边缘机识别后写库 — 流程设计

SmartCabinetLite · 2026-10-21 · V3 · 整体流程 + 重开柜细节 + 异常清单 + 存储设计 + 实施计划

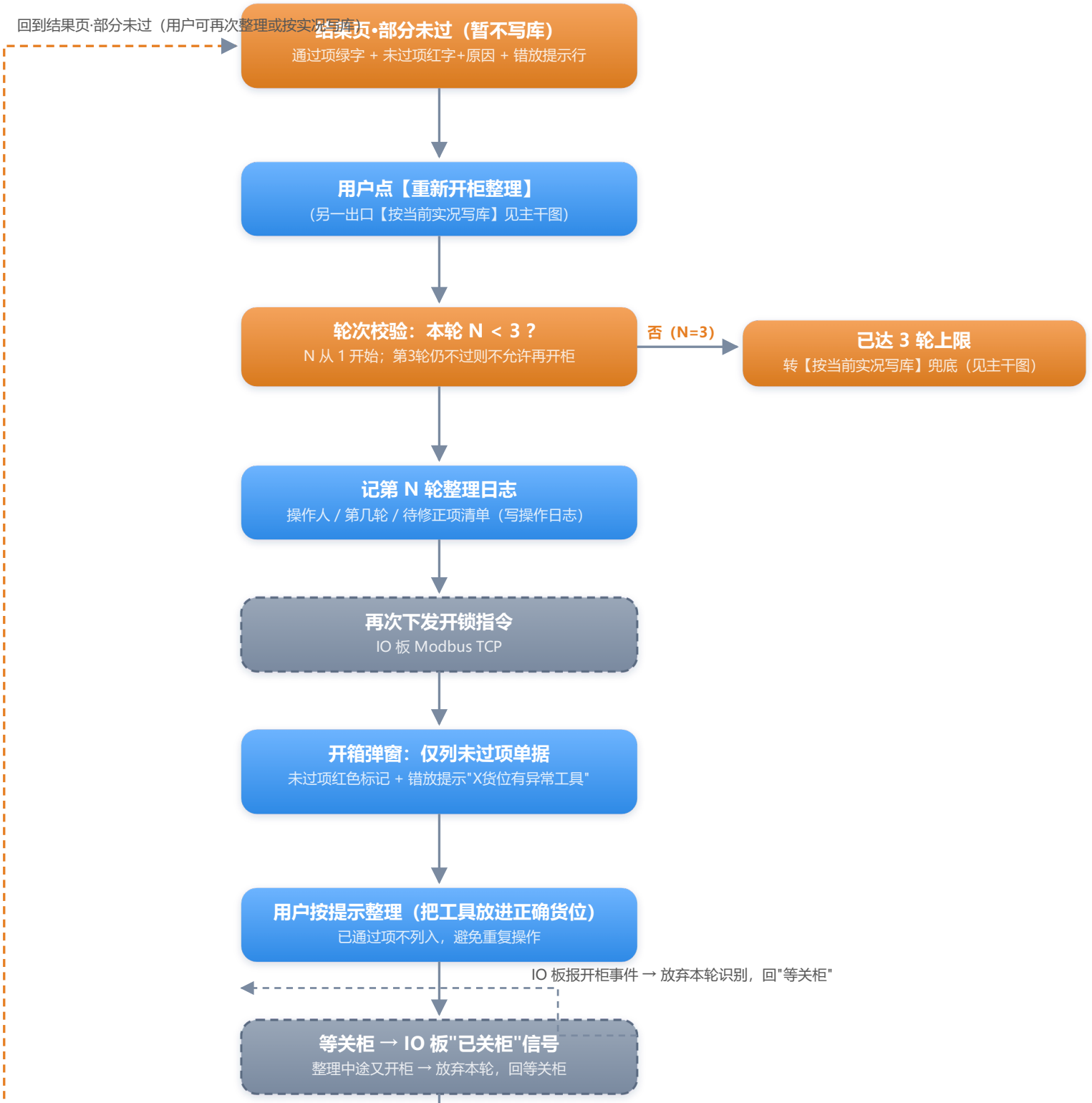
图例 主干流程 成功分支 失败/拒绝分支 等待/超时分支 灰色 = IO 板硬件链路

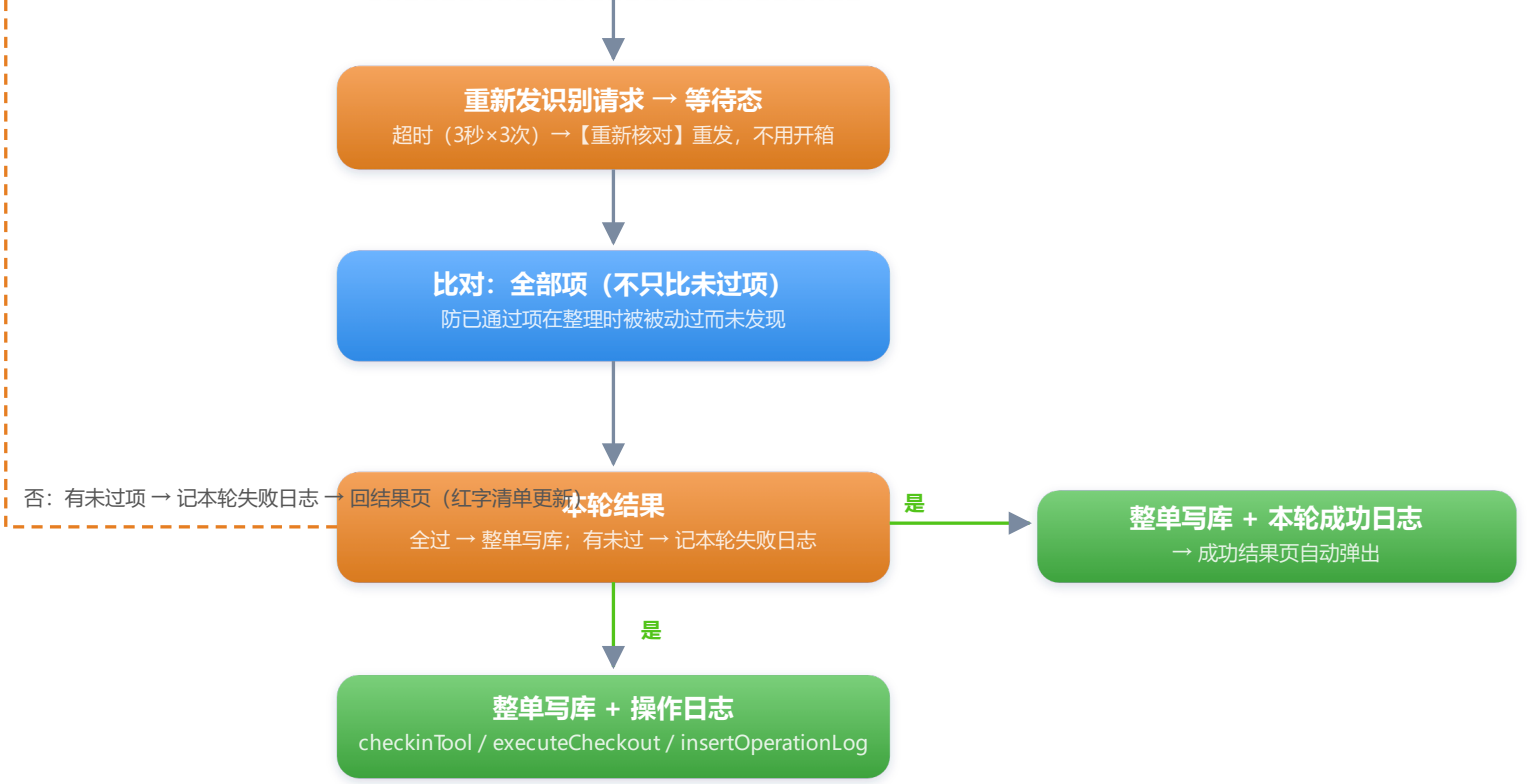
一、主流程图（整体流程，含重开柜闭环与三轮上限）


① 出入库页编辑单据
手动添加 / 一键导入（已带去重与查重）
② 用户点"确认出入库"
m_submitBtn → onSubmit
③ 单据校验
未选工具 / 空单 / 同工具查重 / 位置实时核对
失败
校验失败：红字提示，留在单据界面
（现有逻辑，不进流程）
④ 边缘机链路在线检查
EdgeService.m_online（周期帧15秒判活）
离线
链路离线：拒绝本次，不写库
提示"边缘机链路离线"，禁止出入库
⑤ 下发开锁指令
IO 板 Modbus TCP
⑥ 开箱弹窗：柜体动画 + 单据摘要
提示"请按单据操作并关闭柜门"
⑦ 等待关柜（禁用操作）
提供【取消】放弃本次，零写库
超时
超时未关柜：提示"请确认柜门已关"
可取消，放弃本次零写库
⑧ 收到"已关柜"信号
IO 板门磁上报
重新核对（重发请求）
⑨ 发识别请求 + 等待态
超时
识别超时（3秒×3次）：【重新核对】

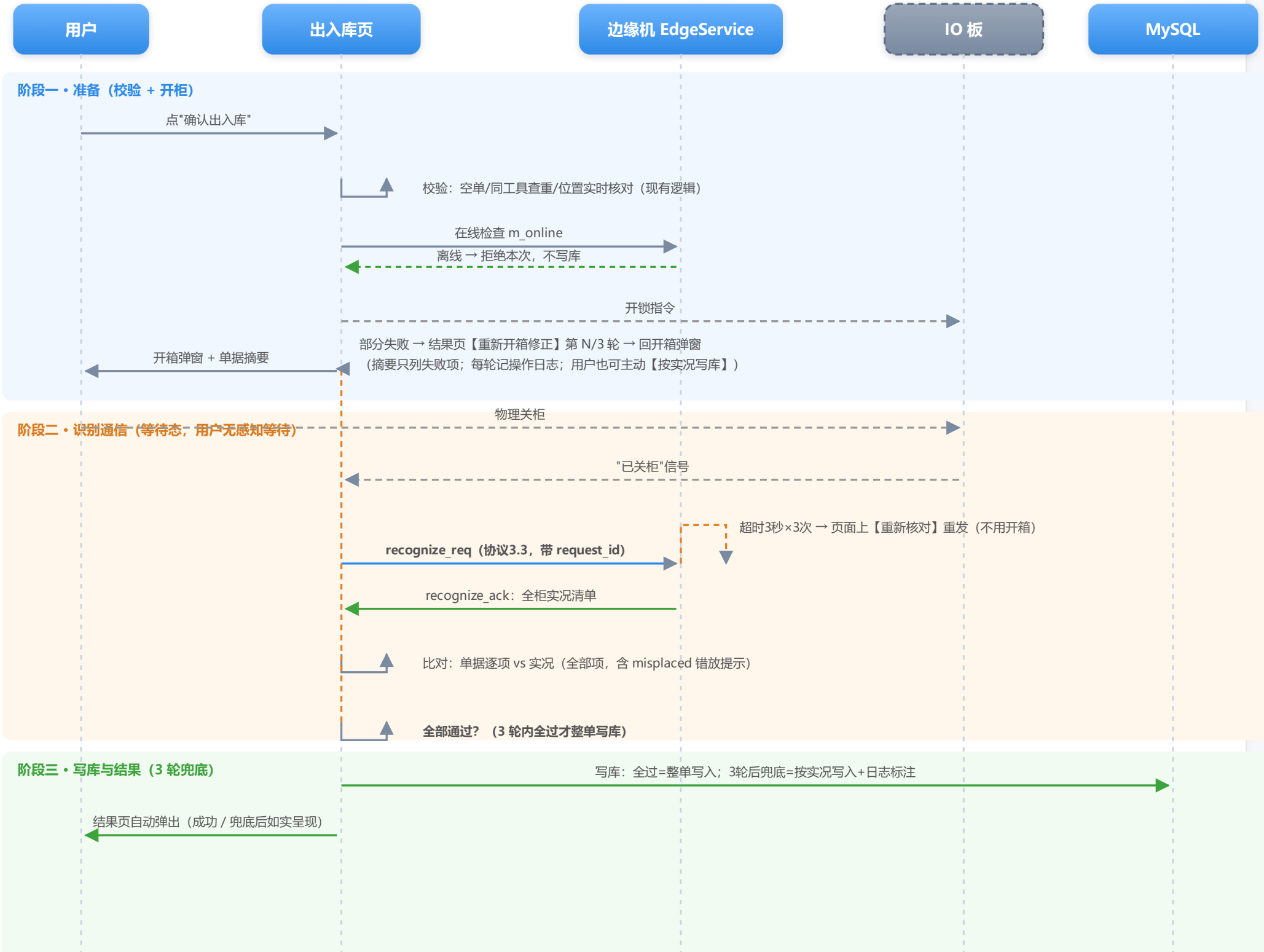


二、重新开柜整理 — 细节流程（袁总点名细化）


回到结果页·部分未过（用户可再次整理或按实况写库）
结果页·部分未过（暂不写库）
通过项绿字 + 未过项红字+原因 + 错放提示行
用户点【重新开柜整理】
（另一出口【按当前实况写库】见主干图）
轮次校验：本轮 N < 3 ？
N 从 1 开始；第3轮仍不过则不允许再开柜
否（N=3）
已达 3 轮上限
转【按当前实况写库】兜底（见主干图）
记第 N 轮整理日志
操作人 / 第几轮 / 待修正项清单（写操作日志）
再次下发开锁指令
IO 板 Modbus TCP
开箱弹窗：仅列未过项单据
未过项红色标记 + 错放提示"X货位有异常工具"
用户按提示整理（把工具放进正确货位）
已通过项不列入，避免重复操作
IO 板报开柜事件 → 放弃本轮识别，回"等关柜"
等关柜 → IO 板"已关柜"信号
整理中途又开柜 → 放弃本轮，回等关柜


重新发识别请求 → 等待态
超时（3秒×3次）→【重新核对】重发，不用开箱
比对：全部项（不只比未过项）
防已通过项在整理时被被动过而未发现
本轮结果
全过 → 整单写库；有未过 → 记本轮失败日志
否：有未过项 → 记本轮失败日志 → 回结果页（红字清单更新）
是
整单写库 + 本轮成功日志
→ 成功结果页自动弹出
是
整单写库 + 操作日志
checkinTool / executeCheckout / insertOperationLog

三、时序图


用户
出入库页
边缘机 EdgeService
IO 板
MySQL
阶段一・准备（校验 + 开柜）
点"确认出入库"
校验：空单/同工具查重/位置实时核对（现有逻辑）
在线检查 m_online
离线 → 拒绝本次，不写库
开锁指令
部分失败 → 结果页【重新开箱修正】第 N/3 轮 → 回开箱弹窗
（摘要只列失败项；每轮记操作日志；用户也可主动【按实况写库】）
开箱弹窗 + 单据摘要
阶段二・识别通信（等待态，用户无感知等待）
物理关柜
"已关柜"信号
超时3秒×3次 → 页面上【重新核对】重发（不用开箱）
recognize_req（协议3.3，带 request_id）
recognize_ack：全柜实况清单
比对：单据逐项 vs 实况（全部项，含 misplaced 错放提示）
全部通过？（3 轮内全过才整单写库）
阶段三・写库与结果（3 轮兜底）
写库：全过=整单写入；3轮后兜底=按实况写入+日志标注
结果页自动弹出（成功 / 兜底后如实呈现）

四、异常流程全清单（穷举分析，24 项）

阶段	#	异常场景	检测点	处理方式	图示状态
A 单据校验 进入流程前	1	行未选工具	collectOrderItems	红字"第X行请选择工具"，留在单据界面	已覆盖
	2	空单据	collectOrderItems	红字"请先添加出入库指令"	已覆盖
	3	同工具多行入库	onSubmit 查重（QSet）	弹窗"已有多行入库指令，请删除多余行"	已覆盖
	4	位置被占/工具信息加载失败	validateOrderItems	弹窗指明原因，留在单据界面	已覆盖
B 通信 流程进行中	5	边缘机链路离线	点确认时查 m_online	拒绝本次，不写库，提示"边缘机链路离线"	已覆盖
	6	关柜信号超时（不关门/IO板故障）	关柜等待计时器	提示"请确认柜门已关"，提供【取消】	已覆盖
	7	识别请求超时（3秒×3次重发全失败）	EdgeService 扫描器	等待态给【重新核对】重发请求，不用开箱	已覆盖
	8	在途请求期间链路中断	EdgeService 登记表	记通信异常，链路恢复按原 request_id 补发一次；页面表现为超时→重新核对	本次标注
	9	坏帧/半帧（JSON解析失败/超长）	EdgeLink 分帧层	链路层丢弃并计数，不上抛；页面无感（等同超时）	本次标注
	10	识别清单为空（边缘机未就绪/摄像头未覆盖）	ack items 为空	全部项目判"未检测到"，进整理闭环（等同部分失败）	本次新增
C 识别结果 比对判定	11	全部通过	比对结果	整单写库 + 操作日志，成功结果页	已覆盖
	12	部分未过（货位空/错工具/低置信）	比对结果	结果页红字+原因，【重新开柜整理】或【按实况写库】	已覆盖

	13	错放 misplaced（A 出现在 B 的货位）	协议 misplaced 告警字段	结果页提示行"X货位有异常工具"，人工修正，软件不自动挪动	已细化
	14	未知物品 unknown（货位上有认不出的东西）	tool_id 为空	提示"X货位有无法识别物品"；若该货位是本单目标位则该行失败	本次新增
	15	在库工具缺失 missing（非本单单据的工具不在位）	missing 告警字段	仅提示不拦本单（超出本单范围，属库存差异）	本次新增
D 用户操作 交互过程	16	等待态点【取消】	取消按钮	放弃本次，零写库，回单据界面	已覆盖
	17	开柜不操作直接关柜	识别结果	目标位全空→全失败→第1轮整理（自然流入闭环）	已覆盖
	18	整理中途又开柜（反悔）	IO 板开柜事件	放弃本轮识别，回"等关柜"	已细化
	19	3 轮仍不全过	轮次计数器	按当前实况写库兜底（不死循环不零写库），日志标注	已覆盖
	20	页面关闭时存在在途识别请求	页面关闭事件	断开 recognizeFinished/Failed 连接 + 清 pendingItems，防回调打到已销毁页面；二期加取消帧	实现要点
E 写库	21	DB 断连/事务失败	service 返回 false	整体提示"写入失败请重试"，内存结果保留可重试写库——系统错不按用户未过处理	本次新增
	22	部分写成功	—	service 单事务保证不发生；checkinTool 一指令一位置，无多位置部分成功	机制保证
F 结果页后	23	结果页开着时他人开柜	—	不影响本次（已完结）；二期可加刷新提示	本次标注
	24	关闭结果页后改单据再确认	—	自然重走全流程（轮次计数器重置）	已覆盖

覆盖度结论：原主干图已覆盖 14 项；本次分析新增/细化 10 项，其中 3 项（#10 识别清单为空、#14 未知物品、#15 在库缺失）是原图未涉及的识别结果分支，#20/#21 是实现层必须处理的防护点。#8/#9/#23 由 EdgeLink/EdgeService 既有机制或时序自然覆盖，只需在图注与代码注释中写明，不必画支线。

五、放错位置的数据存储与状态枚举设计

3.1 设计原则：货位状态机不放 misplaced

"一个位置一个工具"是系统铁律，由 `tool_position_mapping` 唯一键 (cabinet_id, layer, position) 物理保证。 **错放是"一次识别事件"，不是货位的持续状态**——货位状态机保持三值不变： `pending`（待入库）/ `in_stock`（在库）/ `checked_out`（已出库）。 若给货位加 `misplaced` 状态，"一个位置一个工具"的守恒就被打破（位置上既有主人又出现

异物时状态无法自洽），且所有按状态过滤的查询（findAllInStockPositions 等 7 处）都要改，风险远大于收益。

3.2 错放数据的三层落点

落点	本期做法	理由
操作日志 sys_operation_log	每轮识别写一条 `operation_type='checkin_round'/'checkout_round'`，content 为 JSON：`{round, order:[{toolCode,position,result}], misplaced:[{actualPosition,toolCode,expectedOwner}], unknown:[position], confidence}`，result=success/fail	表结构现成（type 是自由字符串、content 是 JSON text），**零改表**；满足"每轮留痕可追溯"
告警 sys_alert + sys_alert_type	**本期不写**（遵袁总前议"先提示"）。字典表已存在 `mismatch`（工具错放）/ `rack_mismatch`（货架错放）/ `missing` 三个现成类型	将来要告警联动时直接启用，**零建表零加类型**
新表 tool_misplacement	**不建**	单柜场景人工修正即可闭环；多柜场合作业队列是将来的独立需求，现在建属过度设计

3.3 本期新增状态枚举（全部走 SC:: 常量，禁裸字符串）

枚举	取值	用途
RecogItemResult（识别结果类型，页面/服务内存态，不入库）	Matched / Empty / WrongTool / LowConfidence / UnknownItem / Misplaced	比对函数 compareOrderWithRecognition 的逐项判定输出，驱动结果页红绿字
RoundState（单据轮次状态，内存态）	Running / Passed / Partial / GivenUp	重开柜闭环状态机，控制按钮可用性与上限
SC::ORDER_MAX_RETRY_ROUNDS	3（红线常量，可调）	重开柜整理轮次上限
SC::RECOG_MIN_CONFIDENCE	0.85	入库项"正确放入"的置信度门槛（低于判 LowConfidence）
SC::ORDER_ROUND_OP_CHECKIN / _CHECKOUT	"checkin_round" / "checkout_round"	轮次日志的 operation_type 值

六、实施计划

一期（本次实施，不依赖 IO 板）

序号	文件	改动内容

1	src/common/Constants.h	新增 ORDER_MAX_RETRY_ROUNDS / RECOG_MIN_CONFIDENCE / ORDER_ROUND_OP_CHECKIN / ORDER_ROUND_OP_CHECKOUT / 等待态与关柜超时常量
2	src/comm/EdgeService.{h,cpp}	新增公开方法 `requestRecognize()`：request_id 生成+登记进 m_pending+发送帧；离线拒发返回空串。超时/重发/补发全复用既有机制，不重复实现
3	src/pages/ToolCheckinPage.{h,cpp}	①onSubmit 拆出 startEdgeCheck（发起+等待态）②新增 onRecognizeFinished/onRecognizeFailed 槽（requestId 配对，非本次请求丢弃）③**compareOrderWithRecognition() 私有纯函数**（单据 vs 实况清单逐项判定，输出 RecogItemResult）④applyOrderResults()（写库+轮次日志+结果页）⑤轮次状态机 m_currentRound ⑥"我已关柜"按钮顶替关柜信号⑦页面关闭清理在途请求（#20）
4	src/pages/ToolCheckinPageDialog.cpp	①开箱弹窗改造：等待态（"正在核对柜内工具…"+取消）②结果页改造：双按钮（重新开柜整理 N/3 / 按实况写库）+ 错放与未知物品提示行
5	src/MainWindow.{h,cpp}	构造 EdgeService 实例并注入 ToolCheckinPage（页面不自己 new，符合分层铁律）
6	src/db/RecordDAO 或 ToolCheckinPage	轮次日志写入（insertOperationLog 复用，type 用新常量，content 组 JSON）
7	tests/	①compareOrderWithRecognition 纯函数单测：6 场景（全过/空位/错工具/低置信/未知物品/错放）②带库 test_services 补充轮次日志断言

无边缘机时的测试策略：现场没有真实边缘机，①"离线拒绝"路径可直接实测（EdgeService 离线→点确认→验证拒绝不写库）②真识别路径用单测 mock——把 RecognizeResult 直接喂给 compareOrderWithRecognition 验证比对与轮次推进，不依赖硬件 ③GUI 冒烟走"离线拒绝"与"我已关柜顶替"路径。

二期（IO 板就绪后）

序号	模块	内容
1	src/comm/IoLink + IoService（新建）	Modbus TCP 客户端，照 EdgeLink/EdgeService 分层范式；CommTypes.h 追加 IoConfig/IoResult
2	开锁指令	IoService 公开 openCabinet()，ToolCheckinPage 调用点替换"直接进入开箱弹窗"
3	关柜信号	IoService doorClosed 信号 → 页面 onCabinetClosed()；删除"我已关柜"顶替按钮
4	借用归还会话接入	CabinetSessionDialog 的内存模拟快照换成 IO 板真实开关柜事件 + 边缘机识别

工作量与验证

一期约 700-900 行（新代码为主、改造集中在 ToolCheckinPage 系列与 EdgeService 一个方法），验证链： 编译 0 错 → ctest 4/4 → arch_guard 全绿 → 带库 test_services → GUI 冒烟（离线拒绝/顶替关柜/等待态/结果页）→ 单测 mock 比对全场景。

七、决策确认清单（袁总逐条确认）

#	事项	结论	状态
1	写库拆"通信→回调写库"异步两段	showCheckResultDialog 拆出 startEdgeCheck / onRecognizeFinished / applyOrderResults	已定
2	边缘机离线/超时/未部署	禁止出入库，提示"边缘机链路离线"后不写库	已定
3	比对逻辑归属	先放 ToolCheckinPage 私有方法，独立成方法预留将来自抽取口	已定
4	开锁指令 / 关柜信号	IO 板 Modbus TCP 通信（新建 IoLink/IoService，照 EdgeLink/EdgeService 分层）	已定
5	识别触发点	IO 板"已关柜"硬件信号触发识别，与软件按钮解耦	已定
6	等待态体验	点确认即发起，等待动画承接耗时，结果回来自动弹结果页，含【取消】	已定
7	写库时机	3 轮内全过则整单写库；3 轮仍不全过（或用户主动选择）则按实况写库：通过项写入、未过项保持原状，日志标注"第3轮未全过，按实况写入"	已定
8	重试上限	收敛为红线常量（如 SC::ORDER_MAX_RETRY_ROUNDS），当前值 3，后续可调	已定
9	错放提示	结果页加"检测到 X 货位有异常工具"提示行，不写告警	已定
10	操作日志	每轮重开箱都记：操作人 / 第几轮 / 修正了哪些项	已定
11	三轮仍不全过的兜底	不死循环、不零写库：按当前实况写库（通过项写入、未过项保持原状），结果页如实呈现，日志标注（袁总 10-21 拍板）	已定
12	重开箱的摘要与比对范围	建议：摘要只列失败项（少误导），比对核对全部项（防已通过项被被动过）	待确认
13	识别超时但柜门已关	建议：等待态给【重新核对】按钮，重发识别请求，不用重新开箱	待确认